# Simplifying the K215 correctness proofs

Two shared commutations, faster checks, and a small transport theorem

Research contribution for Vladimir Reshetnikov's ProveIt project

21 October 2026 UTC

**Abstract**

The best next simplifications are in the machine/source correspondence, not another rewrite of the set-theoretic argument. Two independently investigated source programs have identical control-flow topology; their only primitive differences are adjacent independent increments generated by two shared templates. Reversing those two templates makes their complete ordinal control-flow graphs identical. Alternatively, their original increment-count basic-block graphs already agree. This suggests replacing a general 2,240-entry correspondence certificate with a canonical-site checker reusing the existing bisimulation theorem.

There is also a small immediate implementation opportunity: the site checker still uses linear transition lookup, although the repository already proves a fast trie equivalent to that lookup for bootstrap checking. Reusing this equality avoids inventing a new simulator. For a different 215-state candidate, CM215 has the shortest incremental proof route: transport the existing exact-machine theorem through one local invariant and two three-step blocks. This does not remove the inherited proof dependency closure.

The accompanying code verifies concrete data, graph equalities, local simulation obligations and restricted state-reduction searches. Formal refactor drafts are explicitly uncompiled: neither Lean nor Rocq was available in this environment. No new kernel endpoint or calibrated 214-state machine is claimed.

## Contents

**1 Answers and priorities** **2**

**2 What is already implemented** **2**

**3 The source correspondence is almost a structural identity** **3**

**4 Reduce repeated computation and unnecessary dependencies** **5**

**5 CM215 is the easiest next incremental endpoint** **7**

**6 Other proof-oriented simplifications** **8**

**7 The 214-state question** **10**

**8 Implementation and validation plan** **10**

**9 Sources and reproducibility** **11**

# 1 Answers and priorities

The three questions have different answers.

1. **How to shorten and simplify the existing proofs?** Separate the small mathematical content from repeated path lookup, certificate indexing, and table evaluation. The strongest new structural observation is that two shared increment-commutation templates explain the entire source discrepancy. The lowest-risk computational change is to use the already proved transition trie in site checks and narrow their imports.
2. **Is another 215-state candidate easier to prove? CM215 is the best incremental choice.** Its correctness can inherit the existing endpoint through a small TM-level positive-stuttering argument, with no new proof enumerator, axiom recovery, source-runtime theorem or bootstrap run. There is no demonstrated overall winner for a complete proof rebuilt from scratch. The newly reviewed GC-ZC215 architecture simplifies protected-tail accounting, but still needs a new compiler and scheduler proof. Our shared-pop215 is not recommended as the easiest formalization: it adds a new sentinel compiler and a protected-tail theorem.
3. **Can we presently supply 214 states? No calibrated 214-state construction was obtained.** The restricted searches and invariant analyses below are reproducible negatives, not a general lower bound. A smaller raw allocation is not a consistency machine until its logical compiler and blank initialization are proved.

| Change | Benefit | Current status |
|---|---|---|
| Reuse fast table in site checks | Avoid repeated linear transition lookup | Equality already proved upstream; refactor draft and differential checks here |
| Narrow site-batch imports | Avoid invalidating expensive site proofs after source-proof edits | Static audit and uncompiled patch |
| Normalize two increment templates | Potentially remove the general source certificate | Exact graph equality checked; generic semantic bridge still needed |
| Canonicalize checkpoint paths | Remove 814 redundant path aliases | Exact quotient checked; checker refinement still needed |
| Pack Rocq paths and drop zero offsets | Much smaller generated source | Lossless data and semantic checks; uncompiled Rocq files |
| CM215 endpoint transport | Small new proof for another exact table | Finite local obligations checked; generic kernel theorem still needed |

# 2 What is already implemented

The original literal K215 has full published Lean and Rocq closure evidence at commit `613f9cf28ac`. Its dense-table SHA-256 is

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`.

Those are external proof-assistant results for blank halting iff a finite ZFC refutation. They neither assume consistency as a premise nor constitute an internally encoded PA or ZFC derivation.

The following substantial improvements were already upstream and should not be proposed as new:

- Lean's dominated-implication source fell from 689,857 to 135,417 bytes; Rocq's from 921,691 to 23,678 bytes.
- Lean's AX routine fell from 202,222 to 10,091 bytes by composing a 156-entry word of existing macro contracts.
- Root-iteration and program-transfer inductions were shared, and a general between-candidates seed theorem replaced a seed-specific argument.
- Later main commit `e0b8f9cb8c4`, inspected during this investigation, already contains signed offset-bisimulation symmetry and removes Lean's separately checked reverse certificate. Its source-data module falls from 653,569 to 456,548 bytes. Our earlier zero-offset projection draft is therefore marked *superseded*, not offered as a new improvement.

Phase4 at `5c965ed1600` already goes further: the Lean dominated-implication source is 30,114 bytes, AX 7,563, and the site-coverage facade 14,916. Generic prefix cancellation and shared command-execution lemmas are also implemented. Its recorded current closure has 185 modules, with 10 freshly compiled and 175 hash-bound reused; two typed endpoint clients and ten standard-axiom reports pass. This is incremental validation, not a new full rebuild. Rocq's phase2 dominated proof is 22,673 bytes with 629 nodes and 735 interpreter instructions. These newer figures supersede the earlier phase's sizes as the current baseline.

A targeted freshness check at main `557b6aac99b8` found `KSource215`, `KSites215Batch000`, and `EnumPerm` byte-identical to the retained `e0b8` snapshots. This is not an assertion that the whole repository is unchanged.

The original full builds, later incremental validations, and the latest inspected source changes are distinct evidence. This investigation did not compile the latest closure. Its detailed reports retain historical measurements and exact source pins rather than silently attaching old build results to newer code.

### 2.1 Optimize the real cost, not just the most conspicuous file

The published successful original Lean compilation attempts total 4,485.409 seconds. Their dominant components were:

| Component | Recorded seconds |
|---|---|
| Source equivalence, including data and both directions | 2,291.521 |
| Site checking and coverage | 1,701.227 |
| Bootstrap | 111.374 |
| Original dominated implication proof | 32.731 |
| Original AX routine | 17.509 |

Source and site checking account for 89.02% of that baseline. The now-removed reverse source checks alone cost 1,118.739 seconds, or 24.94%. These are historical component costs, *not measured post-refactor speedups*. A shorter generated proof can even check more slowly if it replaces shared opaque results with a large repeatedly reduced evaluator.

## 3 The source correspondence is almost a structural identity

### 3.1 Exact comparison

The raw program has 4,751 expanded source leaves. Of these, 1,425 are pure control operations: 98 breaks and 1,327 continues. Resolving them leaves 3,326 register-effect/halt occurrences:

$$1,900 \text{ increments} + 1,425 \text{ decrements/tests} + 1 \text{ halt}.$$

The two inspected programs have identical occurrence kinds, initial occurrence and successor ordinals. Their labels differ only by 256 adjacent independent-increment swaps:

| shared structured node | normalization | expanded occurrences |
|---|---|---|
| 188 | $\mathrm{Inc}(6);\mathrm{Inc}(7) \mapsto \mathrm{Inc}(7);\mathrm{Inc}(6)$ | 175 |
| 190 | $\mathrm{Inc}(6);\mathrm{Inc}(1) \mapsto \mathrm{Inc}(1);\mathrm{Inc}(6)$ | 81 |

Changing only those two shared nodes makes the *complete ordered ordinal CFG* equal, including all successor edges. No branch enters the second instruction of a swapped pair except through its first instruction. These are independently executed structural checks on both the literal Lean and Rocq data, not conclusions inferred merely from zero offsets.

### 3.2 Route A: two local contracts and a canonical CFG theorem

For distinct registers, increments commute on every natural-valued register file. Lean already has the relevant update-commutation and sequencing lemmas, and `RMTransfer.closed_of_contract` can lift one changed node's total contract to root-run equivalence. Apply that machinery twice, preserving the old structured program and its calculus proof.

The remaining reusable theorem should connect an executable source program to its canonical ordinal CFG. It must establish:

1. exact correspondence of the initial instruction;
2. exact register effects and both decrement branches;
3. termination of each collapsed control-only segment;
4. preservation and reflection of halt;
5. positive concrete progress between semantic boundaries.

Then finite equality of the two canonical graphs completes the source bridge. A generic compiler must reject an unresolved or cyclic control-only segment rather than silently erasing possible divergence. The observed finite graph equality is evidence for this route, *not yet a kernel proof of that generic theorem.*

### 3.3 Route B: keep both programs and compare basic blocks

The original programs already have identical test/halt basic-block graphs when an edge records the multiset of increments before its next test. Thus one can avoid even the two source rewrites. A canonical-row checker can reuse the existing segment-run and increment-count lemmas; a separate basic-block compiler is an alternative, not a prerequisite.

There are only 1,426 canonical test/halt sites. The current 2,240 certificate entries include 814 redundant representations of the same sites, differing in residual path representation. Rocq already has trailing-zero path-equivalence lemmas; Lean exposes the analogous site boundary. A canonical-path checker can use these results, but must prove alias preservation instead of replacing literal path equality by an unchecked heuristic.

The preferred Lean adapter can retain the existing `RMBisim.Bisim.halts_iff`. Let the relation hold when both raw paths are consumed by `tsite` to a common certified canonical row. A checked row supplies halt or the same tested register and two existing `BranchGood` facts. Unpacking that row gives the existing `Bisim.step` obligation. Existing segment lemmas already account for increment/control execution. This needs new finite-checker soundness adapters and an initial-row proof, but not an entirely new TM semantics library. Merely deleting aliases from the current raw-path `ORel` certificate would be unsound: the relation must change first.

The current general certificate checks 4,479 branches per direction:

$$2 \cdot 2,239 + 1 = 4,479,$$

where the final term is the initial branch. The canonical graph has 2,851 analogous obligations, including its initial edge. This is a reduction in concrete obligations, not a measured kernel runtime improvement.

Further exact labeled-bisimulation minimization reduces the 1,426 canonical sites to only 1,423 classes: three pairs and 1,420 singletons. The quotient preserves the exact tested registers, ordered positive/zero branches, increment-count vectors, and initial edge. This bounded negative favors the simpler canonical-row relation: removing three more rows is unlikely to justify another class-membership layer, while every path still needs binding. It is not a Turing-machine state reduction or a general program-minimality result.

### 3.4 A safer intermediate step

All 2,240 source-certificate offsets are zero. For Rocq, the existing ordinary bisimulation checker has the same path/segment semantics and can avoid the offset extension. For Lean, an older plain decoder supports only 128 iterations while actual certificate paths reach 176 bits. Do not switch to it blindly. Keeping the current 512-iteration forward checker and using its now-implemented signed symmetry is already a safe improvement.

A separate data-tested Rocq prototype stores paths as positive binary integers with a sentinel bit, preserving leading and trailing zeros. It changes the certificate source from 1,499,170 to 178,532 bytes, an 88.09% reduction. Every one of the 4,480 paths and every successor index round-trips exactly. The ordinary checker was independently replayed in Python in both directions. The proposed Rocq files still require parsing, typechecking, kernel evaluation and endpoint assumption checks.

## 4 Reduce repeated computation and unnecessary dependencies

### 4.1 Use the fast table that already exists

Bootstrap already uses a list zipper and balanced transition trie. Replacing its sparse-list interpreter is therefore not a new opportunity. The site checker, however, still evaluates the linear table at the inspected latest snapshot.

The repository's `fast215` theorem already gives pointwise equality between fast lookup and the literal machine table for every natural state and input bit. In Lean, function extensionality turns this into equality of the table functions. Prove the unchanged finite site check using the fast function, then transport the result back using that equality. No new machine semantics, bounded-tape assumption or simulator soundness theorem is necessary. In Rocq, carry pointwise table equality through an evaluator-equivalence induction; do not add a functional-extensionality axiom to endpoints that are currently closed.

Independent linear/trie executions agree throughout all 6,176 guarded site cases and all 2,442,801 symbolic steps in the prototype. They make 1,635,510 table requests. The Python operation model counts 100,886,942 linear list-cell visits versus 13,084,080 trie branches. These are *abstract operation counts*, not Lean benchmark results. The kernel might share or evaluate representations differently; measure actual checking after the small refactor.

The bootstrap was also replayed independently across sparse-list, zipper and bounds-checked bitset models. All 95,714 steps and thirteen exact checkpoints match in state, head and whole-tape

support. The observed head range is $[-18, 199]$. This confirms the concrete implementation data; it is not a reason to replace an already verified bootstrap implementation unnecessarily.

### 4.2 Separate data dependencies from proof dependencies

All nine `KSites215Batch000` through `008` import the full source-equivalence proof although their direct reference is only `KSource215.t215`. Narrow these imports to the data module and retain the old import at the public facade for compatibility. A static name-use audit and patch are included; no compiled success is claimed.

This does not shorten the clean build by the entire site-check time. It avoids invalidating expensive site checks when only the source-equivalence proof changes. The relevant historical batch work is 1,692.447 seconds. A further split should place stable program/tree definitions separately from changing correspondence-certificate data, so even certificate edits do not invalidate the site batches.

### 4.3 Memoize checked meanings, not merely syntax

Concrete latest Lean forward-check profiling finds:

| Query | Calls | Distinct exact queries |
|---|---:|---:|
| Path decoding | 13,438 | 4,332 |
| Program/site lookup | 24,696 | 11,216 |
| Top-level segment evaluation | 8,958 | 5,702 |
| Recursive segment evaluation | 20,216 | 12,520 |

Exact site memoization reduces the modeled tree-node visits from 1,570,586 to 719,771. Long path traversal, not long straight-line segments, is the main duplication: the largest segment has only six site visits. A checked table of unique summaries should carry its exact program, path, guard and fuel/termination premise. Reusing a result from the same hash-consed syntax node alone is unsound.

Indeed the source has only 202 distinct syntax objects (179 internal DAG nodes and 23 leaf constructors) but 4,751 distinct edge/successor-path contexts and 9,501 exact node/prefix contexts. Common-prefix factoring identifies 56 boundary path-update patterns, a promising parameterized proof target, not yet a valid quotient of TM executions.

For generic Rocq certificates that remain after the K215-specific structural simplification, repeated `nth_error` is another avoidable cost. One forward check requires 5,018,706 list-cell inspections for 4,479 successor lookups. A pure inductive 12-bit trie requires 53,748 branch choices. Prove the trie's lookup refinement once; do not introduce an unchecked external lookup oracle or infer wall-clock speedup from these counts.

### 4.4 Share semantic infrastructure across candidates

Thirteen K215 calculus files are identical to K217 after changing only import namespaces: 96,926 bytes and 1,803 lines. Seven copies of the offset-bisimulation module are byte-identical. Move these developments into stable shared modules, retaining thin compatibility facades as needed. Also generalize the candidate-specific 155-line dense-state relabel proof and its hard-coded finite case split.

Keep the final endpoint statements and the useful universal-initial-register-file source theorem unchanged. Source shortening should not quietly weaken theorem scope or add axioms. The

existing large finite certificates are untrusted producers' output checked by proved mechanisms; compression must preserve that boundary.

# 5 CM215 is the easiest next incremental endpoint

CM215 has 215 working states and changes six rows of the verified native K215. Its native SHA-256 is

`e5907f08e0752eada0e25bbf33680341e80e5fe94b52b409aea1ed039edda252`.

Let $K$ be the original machine, $M$ CM215, and

$$\phi(q) = \begin{cases} 52, & q = 176, \\ q, & q \neq 176. \end{cases}$$

The old roles 52 and 176 share the new state 52; physical state 176 becomes an intermediate selector. This is why the construction remains 215 rather than 214.

## 5.1 Replace history reasoning with a local invariant

For reference configurations, use

$$\begin{aligned} q = 52 &\implies \tau(h-1) = 1, \\ q = 156 &\implies \tau(h+1) = 1, \\ q = 176 \ \wedge \ \tau(h) = 0 &\implies \tau(h-1) = 0. \end{aligned}$$

The middle clause makes this a one-step inductive invariant. The only predecessor of state 156 writes 1 and moves left. If state 156 subsequently takes its one branch, it returns onto that written 1, so its exceptional transition to state 176 cannot enter the zero branch. The other incoming edges establish the required predecessor markers directly. Control 0 satisfies the invariant on every initial tape.

The new zero-branch blocks are

$$52 \xrightarrow{0L} 176 \xrightarrow{1R} 129 \xrightarrow{0R} 154, \qquad 52 \xrightarrow{0L} 176 \xrightarrow{0R} 185 \xrightarrow{0R} 168.$$

They exactly simulate old state 52/0 and 176/0 respectively. Both restore the inspected cells and end one cell to the right. All other old transitions take one new step under $\phi$.

Our standalone finite checker validates invariant preservation and these global-effect templates on all 6,840 satisfying state/five-cell-window cases. It also synthesizes a stronger radius-one invariant automatically, requiring only 4,444 accepted cases. The synthesis treats every newly exposed exterior bit nondeterministically; it never assumes it remains blank. Its output is an untrusted certificate, not a theorem by itself.

## 5.2 The small generic theorem

**Theorem 5.1** (Positive stuttering transport)**.** *Suppose machines have total step functions with absorbing halt. Let a source invariant hold initially and be preserved. Assume related configurations have equal heads, extensionally equal tapes, and controls related by a map that preserves and reflects halt. Suppose every source step from a nonhalting configuration is simulated by $k \geq 1$ target steps, reaching a related configuration, with no premature internal halt. Then their related initial configurations are equihalting.*

*Proof.* Extend the block schedule at an already halted source by one absorbing target step. Recursively define the target clock $f(0) = 0$ and $f(n+1) = f(n) + k_n$. Induction gives related configurations at source time $n$ and target time $f(n)$, and $f(n) \geq n$. Source halting at $n$ yields target halting at $f(n)$. Conversely, if the target halts at $t$, absorption makes it halted at $f(t)$. Reflection of halt then makes the source halted at $t$. Thus no inverse state map or new logical completeness proof is required. □

For CM215, $k \in \{1, 3\}$ and the exact clock before halt is $f(n) = n + 2v(n)$, where $v(n)$ counts the two modified zero branches.

Two implementation details are essential. First, the finite checker needs a locality theorem: every checked block reads/writes only inside its supplied window and leaves the exterior untouched. Second, Lean's tape is a finite-support list; writing and restoring may change its list order. Use equality of all tape reads, not literal list equality. Native/dense label bridges must bind the certificate to the exact already verified table and the new canonical table.

Zero-duration stuttering is not enough. A permanently looping source can be related to a target about to halt if every source step is matched by zero target steps. Positivity is what closes the converse direction.

### 5.3 What this does and does not save

The new CM215 endpoint can compose the original K215 theorem with the local transport and state-label bridges. It does not need another 95,714-step bootstrap proof, another 6,176-case runtime proof, or another enumeration/calibration development. It still imports the original proof and its dependencies. Therefore this is a claim about *incremental proof effort*, not the smallest possible foundational development.

For proofs from scratch, R41 is a reasonable controlled decoder benchmark: unchanged command semantics and clean initialization, with maximum address depth 47 reduced to 41. ST215 has a 201-node source but introduces a transfer compiler and protected-tail reasoning. Shared-pop215 adds an even more elaborate sentinel compiler and a three/four-head debt theorem. Smaller source counts alone do not make these proofs easier.

## 6 Other proof-oriented simplifications

### 6.1 GC-ZC215 simplifies resource accounting

The independently supplied proof-simplification/GC-ZC215 packet was reviewed as data and mathematical text; none of its executables was run. Its corrected hybrid is an actual 215-working-state candidate with 430 rows, not merely a source proposal. Its native table SHA-256 is

`4122c18a347028ee6643428b70a8d5c81218d4937d672be3626981d7b50bf503`.

An owned finite audit checks the packet manifest, exact local transition blocks and a whole-tape blank-prefix replay. The final native start is 74. After 117,004 transitions it reaches the command root with $N = 1$, empty residual $L = 0$, and opaque tail $W = 5$. The earlier 69,143-step frame is only intermediate. Its source has 203 distinct subtrees and 4,752 leaves. The claimed 6,177-case arbitrary-natural runtime derivation was *not* independently rerun, and no Lean/Rocq endpoint was compiled.

The ZC scheduler enumerates every raw candidate from zero. For each NIP occurrence in candidate $n$, replacing that occurrence by DROP and re-encoding gives a strictly earlier candidate $m < n$.

Exact parsing preserves the occurrence position; changing that DROP back recovers $n$, giving an injection from pops into earlier pushes. Thus every finite execution prefix has enough protected heads, without an AX batch-size estimate, diagonal-density bound, or positive initial reserve. This is a genuine simplification over the ST/shared-pop resource arguments. It depends on equal push/pop arity, exact monotone encoding, exhaustive increasing enumeration, and preservation of the stack across candidates. The generic uncompiled credit prototype does not discharge those machine-specific premises.

GC additionally changes the ninth AX head's binder source. It removes the counter-loading $J(i)$ step from GEN, but adds a changed AX contract and a gate-nonfalsity obligation. Its marker merge relates the hybrid to a newly allocated 216-state precursor, *not* directly to the verified original K215. Therefore CM215 remains the shortest incremental endpoint. The ZC architecture is a serious proof-oriented alternative, especially for cleaner initialization/resource accounting, but no complete from-scratch proof-size advantage over the original is established.

**Attribution and overlap.** The incoming packet independently proposes offset-bisimulation symmetry, sentinel-bit Rocq path packing, and context-sensitive prefix sharing. These are overlapping prior/parallel contributions, not new discoveries attributable solely to this report. Its 61-line symmetry draft, 190,411-byte compact certificate and 107-line credit prototype are explicitly uncompiled. Lean's signed symmetry is already upstream; our 178,532-byte packing experiment is a separate encoding of the same original certificate. Keep these contributions and their verification status distinct.

### 6.2 A fixed successful candidate suffices at the literal cutoff

A separate simplification applies to the already constructed *common-NIP* command interface, not to GC-ZC215's different opcode assignment. In the completeness direction, assume a finite refutation and let $B_1$ be its compiled word. The established compiler then supplies

$$U = \mathrm{STORE}_0; \mathrm{STORE}_0; B_1; R; R,$$

which halts from every natural command-boundary work-store, possibly before its last command. Its first command is $\mathrm{AX}(0; 0, 0)$, whose common-NIP opcode is 2. Hence its exact positive-continuation code is $P(2, r) \geq 2$ for some natural $r$. The literal initializer leaves next candidate 1 and an empty residual. Ordinary increment-by-one enumeration therefore reaches this one fixed successful candidate unless it has already halted. No varying-parameter $\mathrm{START}_Y$ family, padding construction, or new length lemma is needed for this literal cutoff.

An opcode-independent check uses the already established credit bound $|W(n)| \leq n + 1$ and the exact codec round trip. Since $|\mathrm{STORE}_0| = 10$,

$$\mathrm{code}(U) \geq |U| - 1 = 21 + |B_1| \geq 21.$$

For arbitrary cutoffs, residual domination and positive continuation yield the generic inequality $\mathrm{code}(v) \geq |v| - 1$ for nonempty words. Prefixing a universally halting word by $N$ arbitrary commands then puts its code beyond cutoff $N$, provided each prefix command terminates normally or halts. This does not imply reachability under a skipping scheduler, nor does it replace protected-tail soundness in the opposite direction.

The current phase-four `EnumPerm` is a different four-command enumerator and already has a four-line specialized code bound and nine-line general completeness proof. The generic lemma is a reusable abstraction, not a demonstrated reduction in that current file's source, dependency closure or checking time. The detailed ordinary proof and explicitly uncompiled Lean adapter are in `enumeration/`; no sealed proof or machine was changed.

# 7 The 214-state question

No calibrated 214-state machine was constructed in this investigation. Two complementary restricted searches were completed.

**Local selector synthesis.** A three-row family contains 5,916,800 candidate tables. The initially conservative left-prefix boundary was subsequently extended using the exact known blank-left tape. All trials are classified, including independently replayed cycles; no step cap remains unresolved. No candidate satisfies the chosen first-right-exit and exact-child-state contract. This excludes only that fixed family and contract, not arbitrary 214-state implementations.

**Proof-directed window abstraction.** For the original K215, CM215, ST215 and shared-pop215, least fixed points were computed at radii 1 through 5, both from blank and arbitrary initial windows. All 40 certificates independently pass initial inclusion, inductive closure and leastness checks. Every abstraction retains all 215 working states and 430 read rows; no direct compatible merger or ordinary step-for-step bisimulation merger appears. The largest domain has 440,320 state/window pairs. These are unbounded inductive overapproximations, not simulations for a bounded number of TM steps; nevertheless their failure to remove a state is not a global lower bound.

A separate 200-node common-rotation source did allocate to 214 states under finite screens, but its inference interface has an adjacency invariant that prevents nontrivial detachment from the intended clean proof store. It is quarantined rather than counted as a result. A mathematically repaired interface costs 202 source nodes in the measured family.

The next disciplined search should synthesize small transition rewrites *together with* finite-window transport certificates. Fix a candidate state identification, bound the changed rows and local simulation length, and solve for a table whose local blocks match the reference under a checked invariant. Every survivor must then pass the generic soundness theorem and exact initial/halting bindings. This keeps proof simplification and state reduction aligned without accepting an uncalibrated small table.

# 8 Implementation and validation plan

1. Pin the exact input tables, sources and toolchain versions. Keep the original kernel-checked endpoint available as a regression target.
2. Apply import narrowing and the fast-table site-check refactor first. They preserve existing theorem statements and reuse proved equalities. Rebuild affected modules, print endpoint assumptions, and run a clean closure replay before reporting performance.
3. Prefer the narrow canonical-row adapter: define the relation through exact `tsite` equations, prove the finite checker produces `BranchGood` facts, and reuse existing `Bisim.halts_iff`. A separately proved primitive/basic-block CFG compiler is an alternative route. Retain the original certificate as an independent migration cross-check.
4. Keep certificate packing as a low-risk intermediate option, especially for other candidates whose graphs are not structurally identical. The optional decoded-equality audit may be used during migration without remaining in the production import closure.
5. Implement the generic invariant-restricted positive-stuttering theorem and locality checker, then instantiate CM215. Print assumptions and check the *new exact endpoint*; do not attach old kernel receipts to changed tables.
6. Benchmark clean and incremental builds separately, recording compiler versions, cache state, peak memory, source bytes and proof-object size. Python operation counts are useful

diagnostics, not kernel performance results.

The included scripts and receipts report exactly what was executed. The Lean/Rocq drafts are marked uncompiled. There is no claimed new trusted axiom, proof-assistant endpoint or 214-state result.

## 9 Sources and reproducibility

Detailed source-grounded reports and pinned inputs accompany this article in `lean/`, `rocq/`, `runtime/` and `candidates/`. The independent CM certificate prototype is in `transport/`; the 40 invariant certificates and their independent checker are in `window_reduction/`. The reports distinguish historical measurements from later source changes. The GC-ZC intake assessment is in `new_intake/gc_zc215/`; enumeration arguments and their finite corroboration are in `enumeration/`.

Primary repository references include:

- Full exact-machine verification: `https://github.com/VladimirReshetnikov/ProveIt/blob/613f9cf28ac36eaa929f7ef909b0eb7905facc5d/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/README.md`.
- Published prior simplifications: `https://github.com/VladimirReshetnikov/ProveIt/blob/b6fb232916d008383e7cf2990d68ddb30fded7db/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/README.md`.
- Immutable source snapshot used for the new profiles: `https://github.com/VladimirReshetnikov/ProveIt/commit/e0b8f9cb8c496ff2d6e43f478021fc14d42bbfad`.
- Incoming independent proof-simplification/GC-ZC215 packet: `https://github.com/VladimirReshetnikov/ProveIt/blob/05d063acef885d47be5a2ce6645b046f98f7ea02/docs/incoming/Proof_Simplification_and_GC-ZC215_2026-10-21.zip`.
- CM215's local transition proof: `https://github.com/VladimirReshetnikov/ProveIt/blob/70f50bee5a2a48013645e3c223e9ab7a494fc878/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/conditional-marker/proof/marker215-proof.tex`.

Historical sources, licenses and exact hashes are retained. No external repository was modified during this work.